PCA Factor Strategy

Since financial asset returns are characterized by low signal-to-noise ratio and extreme (tail) events, our goal would be to devise a strategy that: 1. Relies on parsimonious models for any predictions, 2. is conservative enough (e.g. in terms of leverage) so that the negative tail-events do not result in ruin, and 3. diversifies as much as possible within the constraints of the 100 symbols.

For those reasons, we consider a strategy based on the Kelly-Criterion (i.e. maximizing the logarithmic utility function) for the allocation part, and factor model for the predictive part to obtain the expected return and variance inputs to the optimization routine. After transforming the daily close prices into logarithmic returns and splitting the entire dataset into training (2010-2011), validation (2012), and testing (2013) sets, we use the training set to:

- Extract the Principal Components (PCs) and examine the relationship between the in-sample explained variance ratio and the number of components. This helps us decide how many PCs at most we will allow in our factor model
- Select univariate AR(p) models for each of the PCs using the Box-Jenkins approach of examining the ACF/PACF plots, fitting parsimonious AR(p) models, and examining its goodness of fit through looking at the statistical significance of fitted coefficients as well as the distribution and serial dependence of the residuals using the Ljung-Box Q-test
- For each of the 100 symbols, we fit a factor model through regressing the symbol log-returns on the extracted PCs. This allows us to obtain the factor loadings which serve to later forecast returns and estimate variances for the optimization.

Based on tinkering with the models, we decided to use the first 3 PCs, to which we fit AR model with lag 2 as the only lag, AR(1) and AR(2) models respectively. Despite the relative stability of the parameters, we refit the factor loadings and PCs monthly in the backtest (and in our final implementation). As for the optimization, we minimize the objective function:

$$\mathcal{L}(w) = \frac{1}{2} w^\top \Sigma w - \mu_{\text{eff}}^\top w + \lambda \sum_{i=1}^{n} \sqrt{(w_i - w_i^{\text{prev}})^2 + \varepsilon},$$

which is a second-order approximation of the logarithmic utility function with $\lambda = 0.0005$ being the penalty for trading (since we want to consider the transaction costs explicitly), ε being a numeric stabilization constant, and $\mu_{eff} = k * \mu$ being the effective expected return after selecting the kelly-fraction. To stay in the runtime constraints, we limit the number of iterations to 100 which leads to slight differences in results between the same backtest period runs, but these are not significant. We used the estimated covariance matrix and the expected return vector forecast from our models as input to this optimizer for each trading day.

Having all the ingredients needed to run our strategy, we tinker with the rest of the parameters and running back tests over the validation window (2012), we decided to set the Kelly-Fraction to $k = 0.25$, the gross exposure limit to 1.1, since we want to be conservative and only small amount of short-selling allowed leads to the best Sharpe-Ratio in our back tests, This yields an ending log-wealth of about 1.644 or about 417.5% PnL. However, the performance on the testing period 2013 deteriorates to an ending log-wealth of around 0, which suggests either a structural shift in the data-generating process, or we have been noise-trading all along with this strategy, despite its relative parsimony.

AR(1) Trading Algorithm

Background

A time series $\{X_t\}$ satisfies an autoregressive model of order 1 (AR(1)) if it can be decomposed in the following manner:

$$X_t = \alpha_0 + \alpha_1 X_{t-1} + \eta_t,$$

where $\eta_t \sim \text{WN}\left(0, \sigma^2\right)$, and given some initial distribution of X_0. The AR(1) model might be the most parsimonious parametric model that takes into account the temporal dependence inherent to each series. Given the low signal to noise ratio present in financial data, our choice is motivated by the fact that an AR(1) model is less likely to overfit on the data. We first note that an AR(1) model is meaningful from a mathematical point of view if the underlying process is weakly stationary. As a reminder, a process $\{Y_t\}$ is weakly stationary if the mean $\mu_y := \mathbb{E}Y_t$ is constant across time, the variance $\sigma_y^2 = \text{Var}(Y_t)$ is constant across time, and the autocovariance of order h $\gamma_y(h) := \text{Cov}\left(Y_t, Y_{t-h}\right)$ varies only with the lag h, not with the absolute position in time.

Data Description and Transformations

For the purposes of this strategy, we will use the close entry from the `df_train.csv` file. One way to test the stationarity of a time series is to perform an ADF test. We conduct the ADF test on each price series independently, where the null hypothesis asserts that each series contains a unit root. Consistent with empirical observations of financial securities, we were unable to reject the presence of a unit root at 1% significance level for any series. To transform the series into a stationary series, we take the simple returns, defined as

$$R_{i,t} = \frac{\text{Close}_{i,t}}{\text{Close}_{i,t-1}} - 1,$$

where i indexes the individual assets, and t the specific point in time. We performed the ADF test on each return series independently and rejected the presence of a unit root at 1% significance. To test our strategies, we split the entire dataset into a training set (2010-2011), a validation set (2012) and a test set (2013).

Strategy Description

Our AR(1) Trading Algorithm is very simple and intuitive to understand. At time t, we use lagged asset returns $R_{i,t}, R_{i,t-1}, \ldots, R_{i,t-W}, i = 1, 2, \ldots, 100$ to fit an AR(1) model, where $W = 126$ is the fitting window specified by the user. Using the fitted coefficient values, we generate 1-step ahead forecasts $\hat{R}_{i,t+1}, i = 1, 2, \ldots, 100$. Subsequently, we perform a cross-sectional sorting of forecasted returns in ascending order. To translate forecasts into positions, we go long the asset with the largest forecasted return, short the asset with the lowest forecasted return, and assign a weight of 0 to the other assets, with equal wealth spread across the long and short positions. To be precise, let $w_{i,t}$ be the proportion of wealth allocated to asset i at time t. Then

$$w_{i,t} = \begin{cases} 0.5 & \text{if } i = \arg\max_j \hat{R}_{j,t} \\ -0.5 & \text{if } i = \arg\min_j \hat{R}_{j,t} \\ 0 & \text{otherwise.} \end{cases}$$

As a consequence of the above, our portfolio will be very sparse, mitigating the effects of transaction costs on the final wealth. This choice also has an intuitive interpretation: we are more confident in the forecasts at the upper tail and lower tail of the distribution and more uncertain in the forecasts around the middle tail of the distribution.

Results

We tested strategies that go long the assets with the k highest forecasted returns and short to the assets with the k lowest forecasted returns, with $k \in \{1, 5, 10, 20, 25\}$, and different fitting windows $W \in \{21, 42, 63, 126, 252\}$. When testing on the validation window (2012), the strategy with $W = 126$ and $k = 1$ achieved the best performance, with a Sharpe Ratio of 4.012, and a log-wealth of 1.357. This motivated our final choice of hyperparameters for the trading algorithm. On the test set (2013), this strategy achieved a Sharpe Ratio of 2.508, and a log-wealth of 0.816, consolidating our decision to choose this strategy as the final one.

Mean Reversion Strategy

Data Preparation

The dataset provides daily price and volume records for 100 synthetic ETFs spanning four years. Every column was fully filled in with no gaps, so no data cleaning or gap-filling was needed.

To make the data easier to work with, we reshaped it so that each row represents one trading day and each column represents one ETF, giving a 1006×100 price table. From this we computed the daily percentage return for each stock, as well as returns over longer windows to test how well each signal predicts future prices. Trading volume was organised in the same way and used to build one of the candidate signals described below.

Signal Analysis and Model Development

How We Tested Each Signal

We built and tested six different trading signals to find out which one best predicts whether a stock will go up or down the next day. Each signal was scored using two methods:

1. **Information Coefficient (IC):** for each day, we ranked all 100 stocks by their signal value and by their actual next-day return, then measured how well the two rankings agreed (Spearman correlation). A higher IC means the signal is a better predictor. We consider a signal useful if its average IC exceeds 0.02 and its t-statistic exceeds 2 in absolute value.

2. **Simulated portfolio:** we went long the top 20 stocks by signal and short the bottom 20 each day, then tracked the profit after deducting the 5bp transaction cost charged per trade.

We also checked how far ahead each signal can predict by computing the IC at forecast horizons of 1, 2, 3, 5, 10, and 20 days. A signal that stays predictive over many days is more valuable than one that fades after a day.

Results

Signal	Mean IC	t-stat	Avg. Turnover	Net Sharpe (5bp)	Net Ann. Return
MA Cross (5d/20d)	+0.025	4.3	0.52	0.54	+7.9%
Momentum 10d	+0.027	4.8	0.96	0.07	+1.0%
Momentum 20d	+0.001	0.1	—	—	—
MeanRev 60d (contrarian)	**−0.037**	**−7.2**	**0.46**	**2.94**	**+40.2%**
Vol-Weighted Momentum 10d	+0.027	5.3	0.91	0.84	+10.7%

Table 1: Comparison of signals. "Net Sharpe" and "Net Ann. Return" are after 5bp transaction costs.

What we found. The most striking result is the 60-day momentum signal, which turned out to be strongly *negative* (IC $= -0.037$, $t = -7.2$). This means that stocks which have risen over the past 60 days tend to *fall* afterwards, and stocks that have fallen tend to rise. This effect held up consistently in every single year of the training data, showing it is not just a lucky streak in one particular period. The signal also remained predictive out to 20 days ahead, confirming it reflects a genuine recurring pattern rather than random noise. By contrast, the 10-day momentum signal sounds similar but is nearly useless after costs: its high daily trading activity means the 5bp transaction fee eats almost all the profit, leaving a net Sharpe of just 0.07.

Why We Chose This Strategy

We chose the 60-day mean-reversion strategy for three reasons. First, it is by far the strongest and most reliable predictor in the data, with a t-statistic of -7.2 and consistent profitability every year — suggesting this is a real feature of the market rather than a coincidence. Second, because the signal changes slowly, the portfolio does not need to trade very often. Low trading activity means lower costs, and the net Sharpe of 2.94 is barely below the gross Sharpe of 3.36 — a much smaller drop than any other signal. Third, the strategy is simple and transparent: it uses only closing prices with no model to train, no parameters to tune, and no risk of over-engineering the results to look good on historical data.

Signal and Position Sizing

Each day, for every ETF i, we compute:

$$s_{i,t} = -\operatorname{sign}(\text{close}_{i,t} - \text{close}_{i,t-60})$$

If the stock is cheaper than it was 60 days ago, $s_{i,t} = +1$ (buy); if it is more expensive, $s_{i,t} = -1$ (sell). We then spread the current portfolio value W_t equally across all stocks with a non-zero signal:

$$\text{target}_{i,t} = \frac{W_t}{n_{\text{active}}}\, s_{i,t}$$

Because roughly half the stocks are bought and half are sold each day, the portfolio is approximately market-neutral, meaning it does not simply rise and fall with the overall market.

Walk-Forward Result

We trained the strategy on 2010–2012 data and tested it on the completely unseen 2013 period, deducting 5bp costs on every trade.

Metric	Value
Final wealth	1.0837
Log wealth	+0.0804

Starting with \$1, the strategy grew to \$1.08 over one year after all costs — an 8.4% net return on unseen data, confirming that the mean-reversion pattern found in training continues to hold out of sample.

Final Strategy Selection

In our methodology, we have independently tested 3 strategies with the same training, validation, and testing sets. Despite promising results across the board for all 3 presented strategies, only the AR(1) trading algorithm prevailed in the true out-of-sample test on the period 2013, which is why we submit it as our trading strategy of choice.